遞迴、遞迴式與分治

遞迴不只是「函式呼叫自己」。要先說清楚每次呼叫負責什麼、為什麼一定停，再把所有呼叫與合併的成本加起來。

本章依序讀：**一條遞迴路徑 → 合併排序 → 遞迴式的證明工具 → 最大子陣列與矩陣乘法 → 不等分割的延伸工具**。

先備知識：第 1 章的不變量、歸納法、漸近符號與級數；矩陣例子另用基本矩陣乘法，延伸定理使用積分。

參考用書：Cormen 等，*Introduction to Algorithms*, 3rd Edition，Chapters 2、4；另參考 MIT 6.046J（2005）Lecture 2–3。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

子呼叫的答案，是歸納假設的接口

設 $P(n)$ 表示：「對所有大小為 n 的合法輸入，程序會終止，且回傳符合規格的結果。」

1. **基例**：直接核對最小輸入，包括空輸入是否允許。
2. **縮小**：每個子問題都合法，大小嚴格小於 n。
3. **組合**：假設較小呼叫的結果正確，證明目前呼叫也正確。

這是強歸納法。不能先假設「目前這次呼叫正確」，也不能只證子問題變小就當作結果正確。

成本另寫 $T(n) = \sum_i T(n_i) + f(n)$：$f$ 只計本次的比較、分割、複製與合併，**不能把子樹的成本再算一次**。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

二分搜尋只繼續走一邊

前提：$A[1..N]$ 已非遞減排序，鍵值具有全序；回傳**任一**匹配位置，若沒有則回傳 NIL。整體呼叫為 $(1, N)$，$N = 0$ 時直接返回。

```
BINARY-SEARCH(A, low, high, x)
    if low > high: return NIL
    mid = low + floor((high-low)/2)
    if A[mid] == x: return mid
    if A[mid] < x:
        return BINARY-SEARCH(A, mid+1, high, x)
    return BINARY-SEARCH(A, low, mid-1, x)
```

本章採包含兩端點的區間。非空呼叫滿足 $1 \le low \le high \le N$；空區間不讀取陣列。這裡沒有保證回傳第一個重複鍵。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為什麼可以整段排除？

若 $A[mid] < x$，則所有 $i \le mid$ 都有 $A[i] \le A[mid] < x$，所以答案若存在，必在右半。若 $A[mid] > x$，同理只能留左半；相等就已符合規格。

目前長度 $n = high - low + 1 \ge 1$；繼續搜尋的長度至多 $\lfloor n/2 \rfloor < n$。空區間是失敗基例，故遞迴會終止。以長度強歸納，即得整體正確性。

固定成本比較、只傳索引時，最壞成本滿足

$$W(0) = \Theta(1), \qquad W(n) = W(\lfloor n/2 \rfloor) + \Theta(1).$$

沿最長路徑只有 $\Theta(\lg(n+1))$ 個呼叫；最好情況為根即匹配，成本 $\Theta(1)$。順序執行的堆疊峰值為 $\Theta(\lg(n+1))$。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

相同陣列，找到與找不到各走一遍



$A = [3, 5, 7, 8, 9, 12, 15]$。找 9：$(low, mid, high)$ 依序為 $(1, 4, 7)$、$(5, 6, 7)$、$(5, 5, 5)$，回傳 5。

找 10 的前三次檢查相同；最後因 $9 < 10$ 呼叫 $(6, 5)$，空區間回傳 NIL。圖中灰格只表示已排除，不是把原陣列刪掉。

同一個二分邏輯，複製成本可以差很多

固定原始長度為 N，目前區間長度為 n。考慮最壞的搜尋路徑：

傳參方式	每次呼叫的額外工作	整次最壞成本
只傳索引或 view	$\Theta(1)$	$\Theta(\lg(N+1))$
每次複製目前區間	$\Theta(n)$	$\Theta(N)$
每次複製整個原始陣列	$\Theta(N)$	$\Theta(N \lg(N+1))$

第二列沿路求和 $N + N/2 + N/4 + \cdots = \Theta(N)$。第三列中，$N$ 不隨遞迴縮小，不能偷換成 n。

子問題只有一個，也能畫遞迴樹：此時樹退化成一條路。是否適合這個工具，與呼叫數必須大於一無關。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

快速冪：相同子問題只算一次

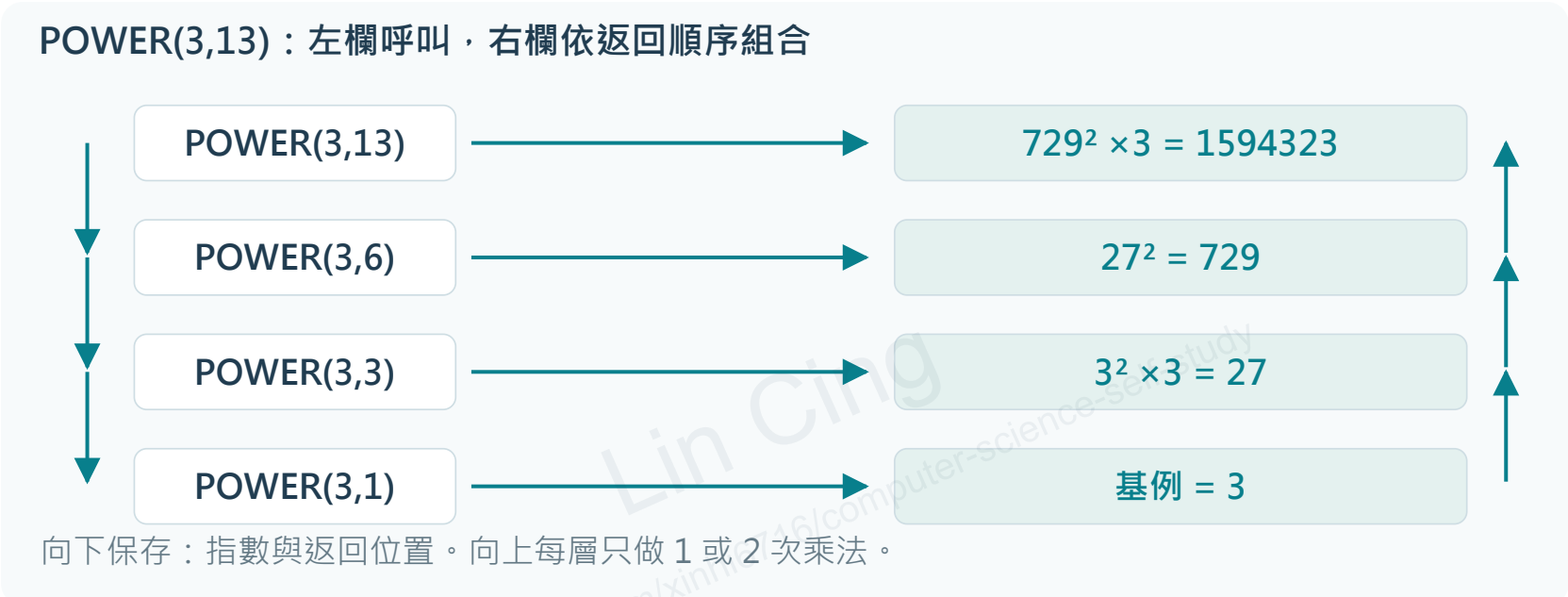
輸入為非負整數 n，輸出 a^n；約定 $a^0 = 1$。本例先計乘法次數，不把大整數乘法一律視為固定時間。

```
POWER(a, n)
    if n == 0: return 1
    if n == 1: return a
    y = POWER(a, floor(n/2))
    z = y*y
    if n is odd: z = z*a
    return z
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

若 $n = 2k$，由 $y = a^k$ 得 $y^2 = a^n$；若 $n = 2k+1$，再乘 a。$\lfloor n/2 \rfloor < n$ 與兩個基例完成強歸納證明。

往下保留返回位置，往上完成乘法



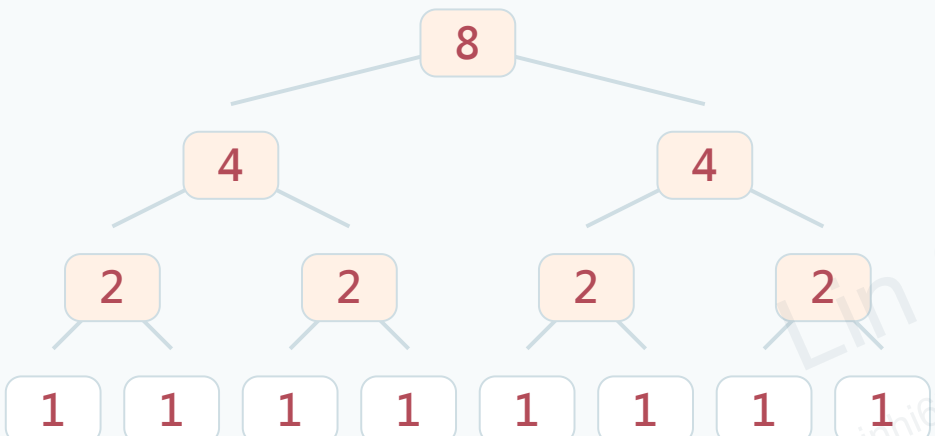
對 3^{13}，向下的指數為 $13 \to 6 \to 3 \to 1$。向上得到 $3 \to 27 \to 729 \to 1594323$；分別做 0、2、1、2 次乘法，共 5 次。

對 $n \geq 1$，平方次數為 $\lfloor \lg n \rfloor$；額外乘 a 的次數為二進位 1 的個數減 1。因此乘法總數為 $\lfloor \lg n \rfloor + \mathrm{popcount}(n) - 1$；$n = 0$ 則為 0。

代數式出現兩次，不代表要呼叫兩次：圖解

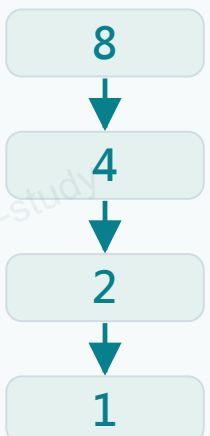
以指數 8 為例，基例指數為 1

每次都重算



15 次呼叫，7 次乘法

保存一次結果



4 次呼叫，3 次乘法

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

代數式出現兩次，不代表要呼叫兩次

若把 $a^{n/2}a^{n/2}$ 寫成兩次獨立的 POWER 呼叫，即使兩次答案相同，成本仍滿足 $D(n) = 2D(n/2) + \Theta(1) = \Theta(n)$，先限 n 為二冪。

保存一次結果再平方，則 $R(n) = R(\lfloor n/2 \rfloor) + \Theta(1)$，在單位成本乘法模型下為 $\Theta(\lg n)$。差別在**實際執行了幾個呼叫**，不是怎麼簡化乘積。

固定寬度實作還要處理乘法溢位；若只需 $a^n \bmod m$，可每次乘法後取模，但乘積本身仍須可安全計算。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fibonacci：呼叫樹的重複與輸出大小

定義 $F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$。直接遞迴每次都重算；呼叫總數滿足 $C_0 = C_1 = 1$、$C_n = C_{n-1} + C_{n-2} + 1$。

令 $D_n = C_n + 1$，由同一遞迴與基例可證 $C_n = 2F_{n+1} - 1$。例如算 $F_5 = 5$ 需要 15 次呼叫；時間為 $\Theta(\varphi^n)$，但堆疊深度只有 $\Theta(n)$，其中 $\varphi = (1 + \sqrt{5})/2$。

由小到大只保存前兩項，做 $n - 1$ 次加法即可算 F_n（$n \geq 2$）。這是避免重算的基本想法，完整動態規劃設計留到後續章節。

算術步數與位元成本要分開：F_n 有 $\Theta(n)$ 個位元；「保存兩個數」是兩個大整數，並非固定數量的機器字。逐項做普通整數加法的位元時間為 $\Theta(n^2)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

用矩陣冪精確計算 Fibonacci

令 $Q = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$。對 $n \geq 1$，

$$Q^n = \begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix}.$$

$n = 1$ 直接成立。右乘 Q 後，第一欄是原本兩欄之和、第二欄是原第一欄，恰好得到下一組 Fibonacci，歸納完成。

例如 $Q^2 = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$、$Q^4 = \begin{bmatrix} 5 & 3 \\ 3 & 2 \end{bmatrix}$，故 $Q^5 = \begin{bmatrix} 8 & 5 \\ 5 & 3 \end{bmatrix}$，右上角為 $F_5 = 5$。$n = 0$ 直接回傳 0。

對矩陣套快速冪，只需 $O(\lg(n+1))$ 次 2×2 矩陣乘法，且全程可用精確整數。這是**算術操作數**；輸出有 $\Theta(n)$ 個位元，因此位元時間不可能是 $O(\lg n)$。用浮點數計閉式再四捨五入也不能無條件保證正確。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

合併排序與分治入門

分治（divide and conquer）把問題拆成較小的同類問題，遞迴求解，再合併答案。以下排序包含兩端點的 $A[p..r]$，長度 $m = r - p + 1$；$p > r$ 表示空區間。

```
MERGE-SORT(A,p,r)
    if p >= r: return
    q = p + floor((r-p)/2)
    MERGE-SORT(A,p,q)
    MERGE-SORT(A,q+1,r)
    MERGE(A,p,q,r)
```

非空呼叫要求 $1 \le p \le r \le n$；整體呼叫 $(1, n)$，空陣列的 $(1, 0)$ 直接返回。子區間長度為 $\lceil m/2 \rceil$、$\lfloor m/2 \rfloor$，$m \ge 2$ 時都嚴格小於 m。

中點寫成差值式，可避免直接計算 $p + r$ 的溢位；索引、長度與加減仍須在實作型別可表示的範圍內。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

不依賴額外鍵值的 MERGE

前提為 $1 \le p \le q < r \le n$，兩半都已穩定排序。L, R 必須是**獨立副本**，長度分別為 $q-p+1$ 與 $r-q$，不能被後續回寫 A 破壞。

```
MERGE(A,p,q,r)
    L = copy(A[p..q]); R = copy(A[q+1..r])
    i = 1; j = 1; k = p
    while i <= L.length and j <= R.length
        if L[i].key <= R[j].key: A[k] = L[i]; i = i+1
        else: A[k] = R[j]; j = j+1
        k = k+1
    while i <= L.length
        A[k] = L[i]; i = i+1; k = k+1
    while j <= R.length
        A[k] = R[j]; j = j+1; k = k+1
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

MERGE 的保存、順序與終止證明

令 $m = r - p + 1$。主迴圈每次比較前，維持

$$k - p = (i - 1) + (j - 1).$$

$A[p..k-1]$ 恰是已消耗的 $L[1..i-1]$ 與 $R[1..j-1]$ 的穩定合併，已排序，且其中每個鍵都不大於任何未消耗的鍵。其餘資料仍在獨立副本中。

初始化時輸出為空，條件成立。兩邊都非空時，各自首筆就是該邊最小的剩餘鍵；取較小者、相等先取左，便是全體下一筆。消耗一筆並令 k 加一，維持長度、保存、順序與穩定性；剩餘筆數嚴格減一，故會終止。

主迴圈結束後，一邊耗盡；另一邊已排序，且不小於輸出尾端，可依序接上。最後 $i = |L| + 1, j = |R| + 1, k = r + 1$，全部 m 筆恰輸出一次。若只用原陣列的 view，回寫可能覆蓋尚未讀取的資料，破壞這個證明的前提。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

有重複 key 的完整合併

令 $L = [1a, 4b, 4c]$、$R = [2d, 4e, 6f]$，左半所有資料原本都早於右半。下表的 i, j 是比較前的索引；只比較鍵值，字母用來辨識資料身分。

i, j	比較與選擇	輸出前綴
1,1	$1 \le 2$，取 1a	1a
2,1	$4 > 2$，取 2d	1a,2d
2,2	$4 \le 4$，取左側 4b	1a,2d,4b
3,2	$4 \le 4$，取左側 4c	1a,2d,4b,4c
4,2	左半耗盡，接上右半	1a,2d,4b,4c,4e,6f

共 4 次鍵值比較、6 次輸出寫入，另有 6 筆資料複製到 L, R。最壞比較數為 $m - 1 = 5$；$\Theta(m)$ 描述成本階，並不表示每次都恰比較 m 次。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

有重複 key 的完整合併：圖解

相等取左：4b、4c 都必須早於 4e

	L			R			輸出前綴
start	1a	4b	4c	2d	4e	6f	∅
1≤2	1a	4b	4c	2d	4e	6f	1a
4>2	1a	4b	4c	2d	4e	6f	1a, 2d
4≤4	1a	4b	4c	2d	4e	6f	1a, 2d, 4b
4≤4	1a	4b	4c	2d	4e	6f	1a, 2d, 4b, 4c
tail	1a	4b	4c	2d	4e	6f	1a, 2d, 4b, 4c, 4e, 6f

綠底 = 已消耗；所有未消耗資料仍保存在獨立的 L、R 副本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sentinel 是有前提的另一種表示

哨兵（sentinel）是在每個暫存陣列尾端加一個特殊標記，讓合併可固定輸出 m 次，不必每輪判斷哪邊耗盡。但標記的排序順位必須**嚴格高於所有合法鍵值**，而且不是可輸出的真資料。

不能直接把最大合法整數當作哨兵。若合法鍵包含 $+\infty$，取 $L = [+\infty]$、$R = [+\infty, +\infty]$，再加相同值的哨兵：等值先取左可能依序取到左邊真資料、左哨兵，第三次便讀超過 L。

本章的長度檢查版適用於任意全序鍵值域。另一個安全做法是使用帶標籤的哨兵，讓它在比較時嚴格高於全部真資料；仍須保證哨兵不會被輸出成答案。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從局部合併推到整個排序

對區間長度 m 作強歸納。$m \le 1$ 時已排序且不需修改。$m \ge 2$ 時，兩個子區間都較短；歸納假設保證兩半各自穩定排序，並保留所有原資料。

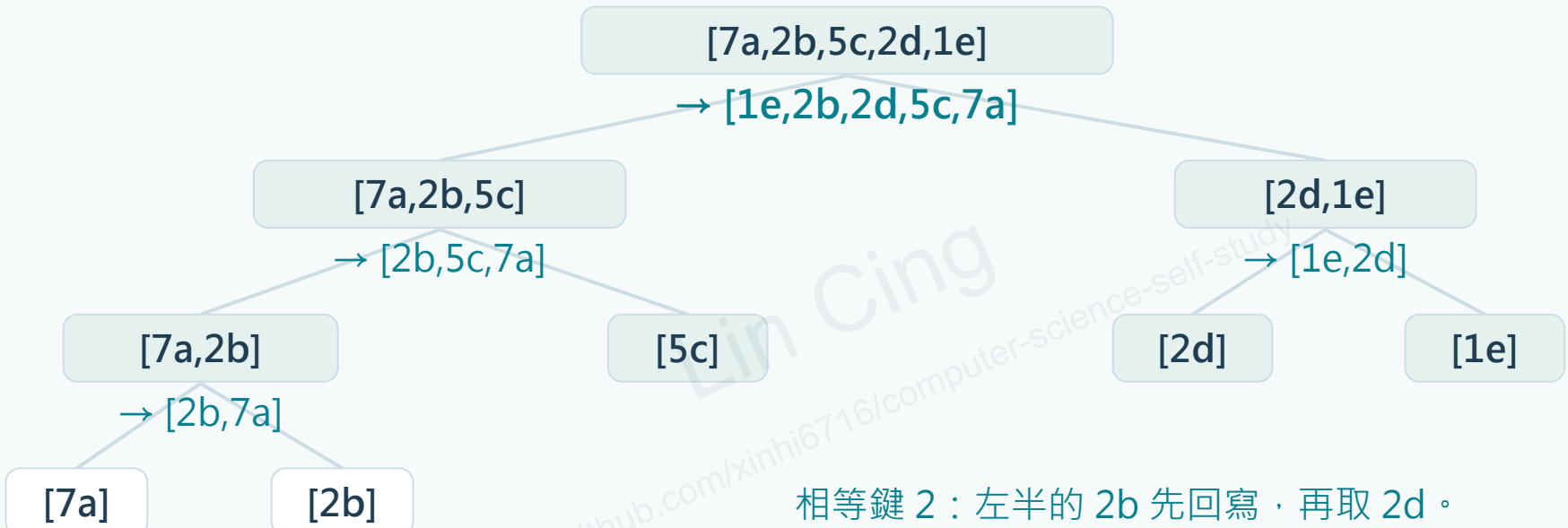
於是 MERGE 的前提成立。合併證明使整段穩定排序且不遺失、不重複資料；區間外未改動，完成目前呼叫的規格。子問題長度嚴格縮小，另保證終止。

此論證不要求 n 是二冪。例如 $[7a, 2b, 5c, 2d, 1e]$ 先分成長 3、2，再分成 2/1 與 1/1。根節點合併遇到相等鍵 2 時，仍先取左邊的 2b，再取 2d。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從局部合併推到整個排序：圖解

往下分割；每個箭頭旁列出該子呼叫的排序結果



相等鍵 2：左半的 2b 先回寫，再取 2d。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

遞迴式的成本不能漏掉合併

對 $m \geq 2$，計算中點為 $\Theta(1)$；兩個子呼叫依序執行，成本相加。MERGE 複製 m 筆並回寫 m 筆，為 $\Theta(m)$，故

$$T(m) = T(\lceil m/2 \rceil) + T(\lfloor m/2 \rfloor) + \Theta(m), \quad T(0), T(1) = \Theta(1).$$

先看二冪 $m = 2^h$ 上的精確代理式 $U(1) = d$、$U(m) = 2U(m/2) + cm$，固定 $c, d > 0$。深度 $\ell < h$ 有 2^ℓ 個內部節點，每個局部成本 $cm/2^\ell$，所以每層總成本 cm。

共有 h 個內部層，最底 m 個葉各花 d，得到

$$U(m) = cm \lg m + dm.$$

真成本不必恰為同一個 c：把各節點的固定上下界分別代入，即得 $\Theta(m \lg m)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

任意長度的界與一個六筆成本表

任意 $m \geq 2$ 平衡拆分後，深度 ℓ 的區間長度不超過 $\lceil m/2^{\ell} \rceil$，所以樹高至多 $\lceil \lg m \rceil$。同層區間互不重疊、總長至多 m，給出 $O(m \lg m)$ 上界。

若 $\ell < \lfloor \lg m \rfloor$，每段至少有 $\lfloor m/2^{\ell} \rfloor \geq 2$ 筆，尚無葉；這些完整層的總長恰為 m。又 $\lfloor \lg m \rfloor \geq \frac{1}{2} \lg m$，故內部工作為 $\Omega(m \lg m)$。另有 m 個單元素基例。

$m = 6$ 的深度	區間長度	仍需合併的長度總和
0	6	6
1	3,3	6
2	2,1,2,1	4
3	1,1,1,1（另兩葉已停止）	0

本例複製到暫存陣列與回寫各 16 筆，基例共 6 次。不能把每一層都算成 6 筆。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

峰值空間不是所有配置次數的總和

本版 MERGE 等兩個子呼叫返回後，才建立總長 m 的 L, R 暫存陣列；完成合併就可釋放。因此子呼叫的暫存陣列不會與父層 MERGE 的暫存陣列同時存活。

根層需要 n 筆暫存資料，其他層不會更大，所以暫存陣列峰值為 $\Theta(n)$。另有 $\Theta(\lg n)$ 個呼叫框架，合計仍為 $\Theta(n)$，這裡取 $n \geq 2$。

全程累計配置量可達 $\Theta(n \lg n)$，但那不是同時需要的記憶體。若保留已完成呼叫的暫存陣列不釋放，就改變了生命期模型；也可預先配置 n 筆 scratch 空間重複使用，但仍須維持未讀取資料不被覆寫的前提。

比較第 1 章：插入排序用常數個額外機器字；此合併排序需要線性額外空間。兩者都可穩定排序，前提分別是嚴格大於才右移、等值時先取左。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

代入法：把猜測證成同一個界

代入法先猜候選界，再用強歸納找出同一組固定常數，使較小輸入的假設推出目前的結論。

令 $T(1) = 1$、$T(n) = 2T(\lfloor n/2 \rfloor) + n$，$n \geq 2$。猜 $T(n) \leq cn \lg n$。對 $n \geq 4$，令 $m = \lfloor n/2 \rfloor \geq 2$，則

$$\begin{aligned} T(n) &\leq 2cm \lg m + n \\ &\leq cn \lg(n/2) + n \\ &= cn \lg n - (c-1)n \leq cn \lg n \end{aligned}$$

只需固定 $c \geq 1$。但 $n = 1$ 的候選右側為 0，不能作歸納基礎。改用遞迴算出的 $T(2) = 4, T(3) = 5$；固定 $c = 2$ 同時覆蓋兩者，而 $n \geq 4$ 的子引數至少為 2，歸納閉合。

遞迴的基例與證明使用的歸納基礎可以不同，但到達小輸入時的依據都必須交代。

LinCing https://github.com/xinhi6716/computer-science-self-study

同一常數的下界與強化假設

先限二冪 n，令 $T(1)=1, T(n)=2T(n/2)+n$。若子問題滿足 $T(n/2)\geq d(n/2)\lg(n/2)$，則

$$T(n)\geq dn\lg n+(1-d)n\geq dn\lg n,\qquad 0<d\leq 1.$$

基例右側為 0、左側為 1，成立。對任意整數的原式 $T(n)=2T(\lfloor n/2\rfloor)+n$，先用歸納證單調：$T(2)\geq T(1)$；其餘相鄰輸入的子引數不下降且較小，故 $T(n+1)\geq T(n)$。令 $m=2^{\lfloor\lg n\rfloor}$，則 $n/2<m\leq n$，由單調性與二冪下界得 $T(n)\geq T(m)\geq dm\lg m=\Omega(n\lg n)$。這裡比較的是同一遞迴式，並非前述 floor／ceil 分割。

再看 $T(n)=T(\lfloor n/2\rfloor)+T(\lceil n/2\rceil)+1$，$T(1)=1$。猜 $T(n)\leq cn$ 只得 $cn+1$，無法閉合；改猜

$$T(n)\leq cn-d.$$

代入得到 $T(n)\leq cn-2d+1\leq cn-d$，只要 $d\geq 1$。取 $c=2, d=1$，基例也成立，並可歸納得到精確解 $2n-1$。

子問題中保留的低階扣項，正好用來支付父節點的新工作。「$cn+n$ 仍是 $O(n)$」卻沒有證出原先同一個 cn，不能作為歸納步驟。

換變數與真正縮小的條件

先在 $n = 2^{2^k}$ 上考慮 $T(n) = 2T(\sqrt{n}) + \lg n$、$T(2) = \Theta(1)$。令 $m = \lg n$、$S(m) = T(2^m)$，則

$$S(m) = 2S(m/2) + m = \Theta(m \lg m).$$

還原得 $T(n) = \Theta(\lg n \lg\lg n)$；不是把新變數 m 又當成原輸入 n。接下來另證帶 floor 的一般整數版本。

固定平移也要檢查是否縮小。例如 $T(n) = 2T(n/2 + 17) + n$，只有 $n > 34$ 時子引數小於 n。令 $n = 34 + x$、$U(x) = T(34 + x)$，得到

$$U(x) = 2U(x/2) + x + 34.$$

採足夠大的固定區間為基例，幾何分層求和得 $\Theta(x \lg x) = \Theta(n \lg n)$。這裡先處理實數引數；整數版還須控制取整誤差。若只給 $T(1)$，$n = 34$ 會自我呼叫，遞迴尚未定義完成。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

平方根裡的 floor，如何補上一般整數證明？

定義 $T(n) = 1$ 於整數 $1 \le n \le 3$，$n \ge 4$ 時

$$T(n) = 2T(\lfloor \sqrt{n} \rfloor) + \lg n.$$

對 $n \ge 4$，$2 \le \lfloor \sqrt{n} \rfloor < n$，所以遞迴有定義且會停止。

先取 $a_k = 2^{2^k}$、$U_k = T(a_k)$。因 $\sqrt{a_k} = a_{k-1}$，這些點沒有取整誤差；$U_0 = T(2) = 1$，且 $U_k = 2U_{k-1} + 2^k$。

除以 2^k，得到 $U_k/2^k = U_{k-1}/2^{k-1} + 1$。累加得 $U_k = (k+1)2^k$，也就是這些特殊點上 $T(n) = \lg n(1 + \lg\lg n)$。

這還只涵蓋特殊點，不能到此就宣稱所有整數的界成立；下一步要補單調性與包夾。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

單調性先證明，再用相鄰特殊點包夾

$T(1)=T(2)=T(3)=1<T(4)$。假設較小引數的相鄰值已非遞減；由 $\lfloor\sqrt{n+1}\rfloor\geq\lfloor\sqrt{n}\rfloor$、兩個子引數均小於 n，以及 $\lg(n+1)>\lg n$，得到 $T(n+1)\geq T(n)$。強歸納證明單調性。

選 k 使 $a_k\leq n<a_{k+1}$，等價於 $2^k\leq\lg n<2^{k+1}$。由單調性，

$$(k+1)2^k\leq T(n)\leq(k+2)2^{k+1}.$$

此區間內 $\lg n$ 與 2^k 在兩倍內，$1+\lg\lg n$ 與 $k+1$ 也在固定倍數內，因此

$$T(n)=\Theta\big(\lg n(1+\lg\lg n)\big),\qquad n\geq 2.$$

對充分大的 n 可寫成 $\Theta(\lg n\lg\lg n)$。這個包夾使用已證的單調性；不是把 floor 的誤差憑直覺丟掉。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

遞迴樹：每個節點只記當次工作

遞迴樹每個內部節點只記本次局部成本 $f(\text{size})$；子呼叫由子樹另計。若節點已記完整 $T(\text{size})$，再加子樹就重複計費。

取 $T(n) = 3T(n/4) + n^2$、$T(1) = 1$，先限 $n = 4^L$。深度 j 有 3^j 個節點，每個局部成本 $(n/4^j)^2$，層成本為 $n^2(3/16)^j$。葉數為 $3^L = n^{\log_4 3}$，故

$$T(n) = n^2 \sum_{j=0}^{L-1} (3/16)^j + n^{\log_4 3}.$$

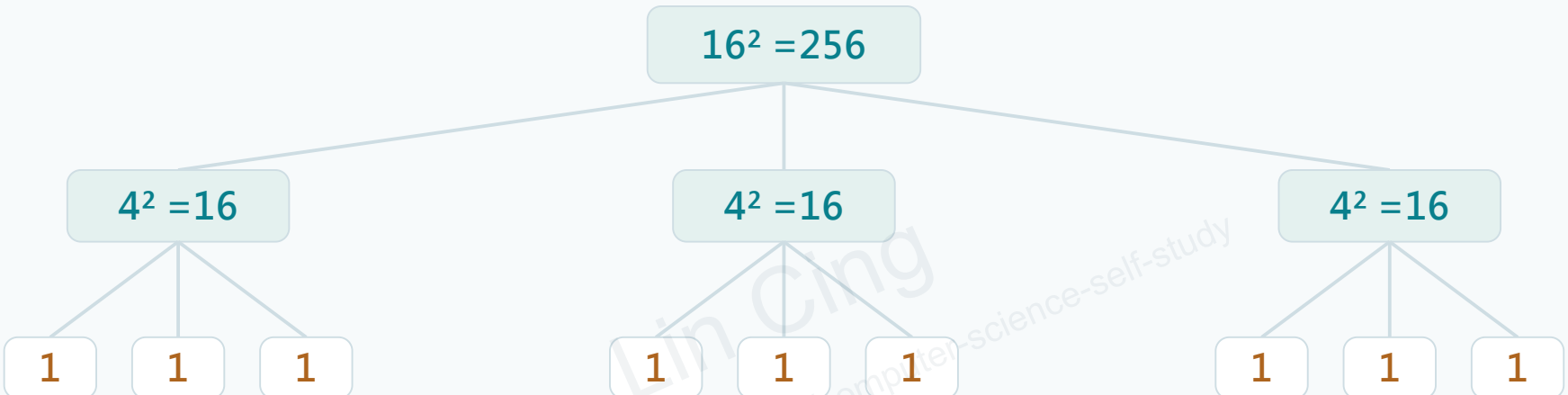
等比和至多 $16/13$，葉成本也不超過 n^2，給出上界 $O(n^2)$；根的局部工作給出下界 $\Omega(n^2)$。

$n = 16$ 的各層成本為 256、48、9，總計 313。最後的 9 已是九個基例的成本，不能再加九次 $f(1)$。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

遞迴樹：每個節點只記當次工作：圖解

$T(n)=3T(n/4)+n^2$，$T(1)=1$；$n=16$



層成本：256 → 48 → 9（九個基例）

總成本 313；葉不再另外加上 f(1)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

每層縮成七分之八，還要把葉成本算進去

取整數 $n \geq 8$，

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lfloor n/4 \rfloor) + T(\lfloor n/8 \rfloor) + n,$$

並令 $0 \leq n \leq 7$ 時 $T(n) = 1$。內部節點的子問題總長至多 $7n/8$，所以深度 j 的內部成本至多 $n(7/8)^j$，合計至多 $8n$。

若內部節點數為 I、葉數為 L，每個內部節點都有三個子節點，數邊得 $3I = I + L - 1$，故 $L = 2I + 1$。每個內部節點至少有成本 8，因而 $8I \leq 8n$，給 $L \leq 2n + 1$。

所以 $T(n) \leq 8n + (2n + 1) = O(n)$；根本身提供 $T(n) \geq n$，得到 $\Theta(n)$。**無窮等比和先支付內部工作，葉成本由節點計數另外支付**。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

不等分割：先界深度，再界每層質量

令 $a = \lfloor n/3 \rfloor$、$b = n - a$，且 $T(n) = T(a) + T(b) + cn$，固定 $c > 0$；小輸入的成本有固定正上下界。

沿任一路徑，第 j 層的引數 q_j 滿足

$$\frac{n}{3^j} - \frac{3}{2} \le q_j \le n(2/3)^j + 3.$$

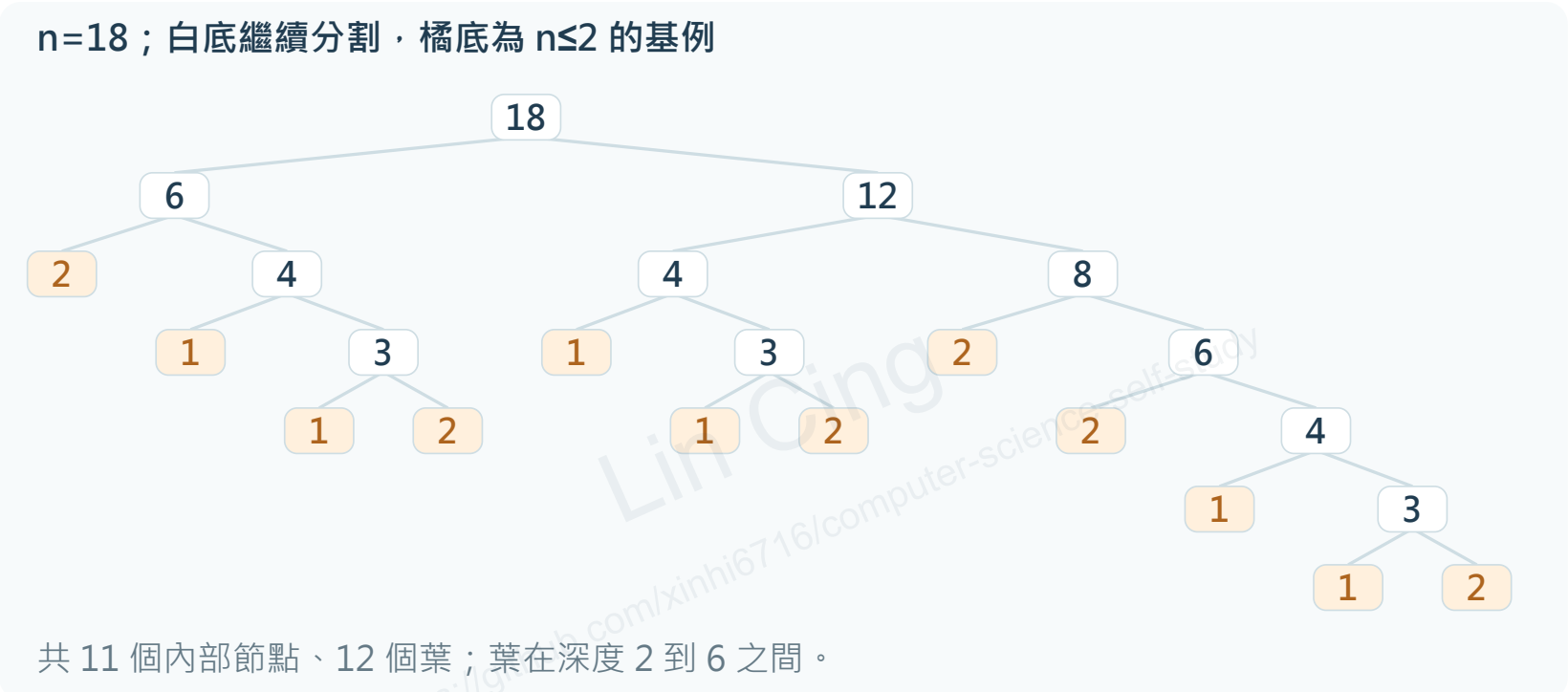
這由每步 $q/3 - 1 \le q_{\text{next}} \le 2q/3 + 1$ 及等比誤差和歸納得到。把大小至多固定 $K \ge 6$ 的整段底層運算視為基例，不改漸近階。右側界給高度 $O(\lg n)$；同層子問題互不重疊，總長度至多 n，工作至多 cn。葉的正整數長度總和是 n，所以至多 n 個葉，合計上界 $O(n \lg n)$。

若 $0 \le j \le \lfloor \log_3(n/(K+2)) \rfloor$，左側界使每個 $q_j > K$，尚無路徑停止。這些完整層都保有總長度 n，各成本 cn，層數為 $\Omega(\lg n)$，所以有相同下界。

若只知局部工作為 $O(n)$，則仍有上界，卻不能保證下界；零合併成本便可能只剩線性葉成本。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

不等分割：完整呼叫樹的葉不在同一層



每條邊是一次實際子呼叫。橘色葉在不同深度停止；最長路徑為18 →12 →8 →6 →4 →3 →2。

不等分割：完整層與提早停止要分清：圖解

n=18；n≤2 停止；條長表示該層內部節點的總量



葉節點成本另計；到達基例後不會繼續產生下一層。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

不等分割：完整層與提早停止要分清

本圖固定 $n \le 2$ 為基例，其餘拆成 $\lfloor n/3 \rfloor$ 與 $n - \lfloor n/3 \rfloor$。$n = 18$ 的完整層總量起初都是 18；出現葉之後，**只有仍繼續分割的節點**才支付合併工作。

最短路徑決定下界能保留幾層；最長路徑決定上界要涵蓋幾層。不能用最深路徑的高度，乘上每一層都等於 n 的假想成本來證下界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先看每層工作，再看三種主導方式：圖解

各欄採獨立比例；標籤為每層總成本



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先看每層工作，再看三種主導方式

三例都取 $n = 16$、$a = b = 2$、$T(1) = 1$。非遞迴項依序為 $1, n, n^2$，各層（含葉）的成本為：

$f(n)$	深度 0、1、2、3、4	總成本
1	1、2、4、8、16	31
n	16、16、16、16、16	80
n^2	256、128、64、32、16	496

這三個數例分別呈現葉端、等高、根端主導；一般定理還需要固定參數、多項式差距與情形 3 的正則性，不能只靠圖的外觀判定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

主定理：先核對前提

令固定 $a \geq 1, b > 1$，局部成本 $f \geq 0$，常數大小基例的成本有固定正上下界。考慮

$$T(n) = aT(n/b) + f(n), \qquad p = \log_b a.$$

先在精確冪上使用下表；取整的延伸條件在後面說明。若 a 是實際呼叫數，它必須為整數；實數 a 可解讀為遞迴權重。

情形	需要核對的前提	結論
1	固定 $\epsilon > 0$，$f(n) = O(n^{p-\epsilon})$	$\Theta(n^p)$
2	$f(n) = \Theta(n^p)$	$\Theta(n^p \lg n)$
3	固定 $\epsilon > 0$，$f(n) = \Omega(n^{p+\epsilon})$；且充分大 n 有 $af(n/b) \leq cf(n)$，固定 $c < 1$	$\Theta(f(n))$

只說「比 n^p 小／大」並不夠：情形 1、3 要有固定的多項式差距，情形 3 還要檢查正則性（regularity）。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一張表的完整判定數例

每一列都使用固定正基例。表中的 p 是 $\log_b a$，不是從圖上猜出的指數。

$(a, b, f(n))$	前提的具體核對	緊界
$(9, 3, n)$	$p = 2$，取 $\epsilon = 1$	$\Theta(n^2)$
$(1, 3/2, 1)$	$p = 0$，$f(n) = \Theta(n^0)$	$\Theta(\lg n)$
$(3, 4, n \lg n)$	$p \approx 0.7925$；取 $\epsilon = 0.1$，正則性比值至多 $3/4$	$\Theta(n \lg n)$
$(8, 2, n^2)$	$p = 3$，取 $\epsilon = 1$	$\Theta(n^3)$
$(7, 2, n^2)$	$p = \lg 7$，取 $\epsilon = \lg 7 - 2 > 0$	$\Theta(n^{\lg 7})$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

核對正則性，再檢查適用邊界

第三列的正則性由

$$\frac{3f(n/4)}{f(n)} = \frac{3}{4}\frac{\lg n - 2}{\lg n} \le \frac{3}{4} \quad (n \ge 4)$$

直接得到。相較之下，$2T(n/2) + n\lg n$ 的對數因子沒有固定多項式差距，不能直接套經典三情形；下一個層和仍能解它。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

對數邊界必須真的求和

令 $n = b^L, p = \log_b a$，且 $f(n) = \Theta(n^p (\lg n)^k)$，固定實數 k。在固定大小的基例以上，把深度 j 改編成離葉端的距離 $r = L - j$，層成本為 $\Theta(n^p r^k)$。

所以總成本是 $\Theta(n^p)$ 的葉成本，加上 n^p 乘一個冪次和：

$$T(n) = \begin{cases} \Theta(n^p (\lg n)^{k+1}) & k > -1, \\ \Theta(n^p \lg \lg n) & k = -1, \\ \Theta(n^p) & k < -1. \end{cases}$$

當 $k > -1$，積分比較給 $\sum_{r=1}^{L} r^k = \Theta(L^{k+1})$；$k = -1$ 是調和和；$k < -1$ 時和收斂到正常數。若 $k < 0$，須在大於 1 的固定基例之外使用公式，避免對 $\lg 1$ 取負次方。

因此 $2T(n/2) + n/\lg n$ 為 $\Theta(n \lg \lg n)$；換成 $n/(\lg n)^2$ 則為 $\Theta(n)$。這是層和的結論，不是經典三情形中的情形 1。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

正則性缺失的明確反例

只看 $n = 2^L$，令 $T(1) = 1$，且

$$f(n) = \begin{cases} n^2 & L \text{ 為偶數}, \\ n^3 & L \text{ 為奇數}, \end{cases} \qquad T(n) = 2T(n/2) + f(n).$$

每個輸入都有 $f(n) \geq n^2$，對 $a = b = 2, p = 1$ 已有固定多項式差距。但在偶數 $L \geq 2$ 上，子問題的指數 $L - 1$ 是奇數，故

$$T(n) \geq 2f(n/2) = n^3/4, \qquad \frac{T(n)}{f(n)} \geq n/4 \to \infty.$$

所以 $T(n)$ 不屬於 $\Theta(f(n))$。同時計算得 $2f(n/2)/f(n) = n/4$，不存在共同的 $c < 1$ 讓正則性最終成立。

這個例子說明缺少前提會怎樣，沒有推翻主定理。對振盪函數也不能因引數只差一個常數，就假設函數值接近。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

主定理為什麼成立

先令 $n = b^L, T(1) = t > 0$。展開 L 次可得

$$T(n) = tn^p + \sum_{j=0}^{L-1} a^j f(n/b^j).$$

第 j 層總權重是 a^j；a 為整數呼叫數時，它也是節點數。葉項 tn^p 在三種情形中都提供正下界。

情形 1：充分大 x 有 $f(x) \le Cx^{p-\epsilon}$，因此大規模層的總和至多

$$Cn^{p-\epsilon} \sum_{j=0}^{L-1} (b^\epsilon)^j = O(n^{p-\epsilon} b^{\epsilon L}) = O(n^p).$$

因 $b^\epsilon > 1$，這是往葉端增大的幾何和。上式可補上任意有限個小引數項來作上界；這些底層的總權重為 $O(n^p)$，固定函數值可吸收到常數。

加上葉成本得到 $O(n^p)$，再用 tn^p 的下界，證得 $\Theta(n^p)$。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

主定理等高與根主導的兩個證明

情形 2。 因 $a = b^p$，充分大引數的每層總成本為

$$a^j \Theta((n/b^j)^p) = \Theta(n^p).$$

有 $L - O(1) = \Theta(\lg n)$ 個這樣的層；將上下界分別相加，得到 $\Theta(n^p \lg n)$，也涵蓋葉成本。

情形 3。 正則性可以逐層使用：只要引數仍充分大，就有

$$a^j f(n/b^j) \le c^j f(n), \qquad 0 < c < 1.$$

這些層合計不超過 $f(n)/(1-c)$。剩餘固定大小層與葉的總成本為 $O(n^p)$；因 $f(n) = \Omega(n^{p+\epsilon})$，也屬於 $O(f(n))$。根的局部工作 $f(n)$ 給出下界，合成 $\Theta(f(n))$。

固定小輸入不是整棵樹的常數成本：可能有 $\Theta(n^p)$ 個小節點，必須像這裡一樣計入底層總量。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

取整的誤差不會逐層線性累積

沿任一實際呼叫路徑，令 $q_0 = n$，q_{j+1} 為 $\lfloor q_j / b \rfloor$ 或 $\lceil q_j / b \rceil$。兩者都有 $|q_{j+1} - q_j / b| \le 1$。取固定 $C \ge b/(b-1)$，歸納得

$$|q_j - n/b^j| \le C.$$

因下一步誤差至多 $C/b + 1 \le C$，取整誤差不會每層累加成 j。

選固定基例範圍 $q \le n_0$，其中 $n_0 > C + 2$，且足夠大使子問題嚴格縮小。上、下誤差界使各路徑進入基例的深度都為 $\log_b n + O(1)$。在 $n/b^j \ge 2C$ 的大規模層，實際 q_j 與 n/b^j 在固定倍數內。

實際有整數 a 個子呼叫時，葉深度只差常數，故葉數為 $\Theta(a^{\log_b n}) = \Theta(n^p)$；固定個底層的總成本為 $O(n^p)$。$a = 1$ 時是一條路，葉與固定個底層的成本都是常數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從精確冪推到所有整數的成本條件

以下處理整數 a 個實際子呼叫；精確冪上的實數權重版本已由前面的展開式處理。

情形 1、2： 大規模層的 q_j 與 n/b^j 在固定倍數內。固定次方即使為負，仍有

$$q_j^{p-\epsilon} = \Theta((n/b^j)^{p-\epsilon}).$$

因此情形 1 可沿用多項式上界與幾何和；情形 2 的每個完整大規模層仍為 $\Theta(n^p)$，層數為 $\Theta(\lg n)$。固定個不完整底層另計 $O(n^p)$。對數擴充在遠離固定基例的層，$\log q_j$ 與離底層深度同階，也可沿用冪次和。

情形 3： 正則性必須涵蓋實際取整後的引數。充分條件是

$$a \max\{f(\lfloor x/b \rfloor), f(\lceil x/b \rceil)\} \le c f(x), \qquad c < 1.$$

各層總成本才會以 c 倍收縮；底層仍由多項式差距吸收。對任意 f，只證引數相差常數並不足以省略此條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

正則性也能推出多項式分離

在精確冪上，若 $af(x/b) \le cf(x)$、固定 $0 < c < 1$，且某個固定底層點 $f(x_0) > 0$，向上反推 k 層可得

$$f(b^k x_0) \ge (a/c)^k f(x_0).$$

因 $\log_b(a/c) = p + \log_b(1/c)$，令 $\epsilon = \log_b(1/c) > 0$，便有沿這條精確冪序列的 $f(n) = \Omega(n^{p+\epsilon})$。

這解釋為什麼有些版本在正且良好的底層條件下，不另外列出多項式差距。本章保留三情形的完整假設，方便逐項核對。

此推導的域是 $n = b^k x_0$；推到全部實數或整數，還需控制最後進入的固定區間，以及該區間上 f 的正下界。不能只證一條序列，就宣稱所有輸入都成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最大非空連續子陣列

給定 $n \geq 1$ 個有限實數 $A[1..n]$，求非空、連續區間 $[i, j]$，使

$$\sum_{k=i}^{j} A[k]$$

最大，回傳 (i, j, sum)。相同最大值可以有多個答案；此規格允許任選其中一個。先以精確加法與比較的操作數分析；浮點誤差、NaN 與溢位不是這個模型的合法輸入行為。

若每天價格為 $P_0, \ldots, P_n$，令 $A_i = P_i - P_{i-1}$，便有 $P_j - P_{i-1} = \sum_{k=i}^{j} A_k$。區間 $i..j$ 對應第 $i-1$ 天買、第 j 天賣。

若允許不交易，才可加入空區間、收益 0；全負輸入時答案會改變。逐一起點、延長終點並累加，可在 $\Theta(n^2)$ 時間與 $O(1)$ 額外空間枚舉；每個區間若重新加總，才變成 $\Theta(n^3)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

跨中線必須由 後綴 與 前綴 組成

對 $low < high$，令 $mid = low + \lfloor (high - low)/2 \rfloor$。任一非空區間恰屬三類：

1. 全在 $low..mid$。
2. 全在 $mid + 1..high$。
3. 跨過中線，滿足 $i \le mid < j$。

第三類必包含 mid 與 $mid + 1$，可唯一拆成左半的後綴（suffix）$i..mid$ 與右半的前綴（prefix）$mid + 1..j$。

兩側端點可獨立選擇，故最佳跨線和等於**最佳非空左後綴和＋最佳非空右前綴和**。分別掃描兩側即可，不必枚舉全部端點配對。左右兩半各自的最佳子陣列則未必接到中線，不能直接相加。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

跨中線必須由 後綴 與 前綴 組成：圖解

mid=5：跨線區間必須包含第 5 與第 6 格



左側向外累加：−1, 3, 0, 1, −1 → 最大 3，起點 4

右側向外累加：2, 3, −2, 2 → 最大 3，終點 7

組合為 A[4..7]=[4,−1,2,1]，總和 6。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

跨線掃描的完整程序

```
CROSS(A, low, mid, high):
    left = -infinity; total = 0
    for i = mid downto low:
        total += A[i]
        if total > left: left = total; best_i = i
    right = -infinity; total = 0
    for j = mid+1 to high:
        total += A[j]
        if total > right: right = total; best_j = j
    return (best_i, best_j, left+right)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

跨線掃描的不變量與成本

每次左迴圈更新後，`total` 恰是 $A[i..mid]$ 的和；`left` 是已掃過全部左後綴的最大和。初值 $-\infty$ 是比較標記，不是輸入資料；第一個有限候選一定更新它，並設定起點。

每輪新增恰一個候選、取最大，保持不變量；結束時左後綴已全部納入。右邊的前綴同理，兩者相加得到最佳跨線和。掃描總次數為

$$(mid - low + 1) + (high - mid) = high - low + 1.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最大子陣列的遞迴與一般證明

```
MAX-SUBARRAY(A, low, high):
    if low == high: return (low, high, A[low])
    mid = low + floor((high-low)/2)
    L = MAX-SUBARRAY(A, low, mid)
    R = MAX-SUBARRAY(A, mid+1, high)
    C = CROSS(A, low, mid, high)
    return a tuple with largest sum among L, R, C
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

遞迴解的正確性與成本

對區間長度作強歸納。單元素只有自身；較長時兩個子區間均非空且較短，子呼叫分別給前兩類的最大值，CROSS 給第三類最大值。三類互斥且完整，取三者最大即正確；長度嚴格縮小也保證終止。

因此操作成本滿足

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \Theta(n) = \Theta(n \lg n).$$

依序執行子呼叫、只傳索引時，每層僅保存常數個索引與和，堆疊峰值為 $O(\lg n)$。若額外複製子陣列，必須另算配置與存活空間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

九元素數例：每次跨線比較都可核對

對 $A = [-2, 1, -3, 4, -1, 2, 1, -5, 4]$，取 $low = 1, high = 9, mid = 5$。

左索引 i	$A[i..5]$ 累計和	已知最佳和、起點
5	-1	-1，5
4	3	3，4
3	0	3，4
2	1	3，4
1	-1	3，4

右邊 $j = 6, 7, 8, 9$ 的前綴和是 $2, 3, -2, 2$，最佳為 $6..7$、和 3。跨線解為 $4..7$、和 6；左半最佳 $4..4$、和 4，右半最佳 $9..9$、和 4，所以本次回傳 $(4, 7, 6)$。

對全負輸入 $[-8, -2, -5]$，非空規格回傳 $(2, 2, -2)$，不是 0。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

線性方法的狀態與不變量

令 E_i 為必須以 i 結尾的最佳非空區間和，B_i 為 $1..i$ 中最佳非空區間和。則

$$E_1 = B_1 = A_1, \qquad E_i = \max(A_i, E_{i-1} + A_i), \quad B_i = \max(B_{i-1}, E_i).$$

以 i 結尾的區間不是只有 A_i，就是接在以 $i-1$ 結尾的區間後；要延伸時，選先前最佳便足夠。這個分類與基例給出兩個狀態的歸納證明。

若選單元素 A_i，起點重設為 i；若選延伸，保留舊起點。全域最佳改善時保存該起點與終點 i。每輪固定工作，故時間 $\Theta(n)$、額外空間 $O(1)$。

九元素例的 E_i 為 $[-2, 1, -2, 4, 3, 5, 6, 1, 5]$，$B_i$ 為 $[-2, 1, 1, 4, 4, 5, 6, 6, 6]$。分治法不是這題最快的解法；它提供的是另一個清楚的分類與組合證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從純量乘積到區塊矩陣乘法

標準矩陣乘法逐一計算

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}.$$

內迴圈在處理第 k 項後，累積值恰為前 k 個乘積之和。n^2 個輸出各做 n 次乘加，算術操作數為 $\Theta(n^3)$；這沒有證明所有矩陣乘法都必須是立方成本。

對偶數階矩陣，寫成區塊

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, \qquad B = \begin{bmatrix} e & f \\ g & h \end{bmatrix}.$$

每個字母是 $n/2$ 階矩陣，乘法給出

$$C_{11} = ae + bg, \quad C_{12} = af + bh, \quad C_{21} = ce + dg, \quad C_{22} = cf + dh.$$

這裡保持乘法次序；矩陣區塊一般不交換。

不能把八個子呼叫吸收到一個子呼叫的常數裡。第 j 層有 8^j 次呼叫，葉數是 $8^{\lg n} = n^3$；分支數會改變整棵樹的成本指數。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

八次遞迴乘法的完整成本

先補零到二冪階數，最後擷取原尺寸結果。每次呼叫從自己的矩陣 view 讀出階數，不能一直沿用原始階數。

```
BLOCK-MULTIPLY(A,B):
    n = A.rows
    if n == 1: return [[A[1,1]*B[1,1]]]
    view A as [[a,b],[c,d]], B as [[e,f],[g,h]]
    C11 = BLOCK-MULTIPLY(a,e)+BLOCK-MULTIPLY(b,g)
    C12 = BLOCK-MULTIPLY(a,f)+BLOCK-MULTIPLY(b,h)
    C21 = BLOCK-MULTIPLY(c,e)+BLOCK-MULTIPLY(d,g)
    C22 = BLOCK-MULTIPLY(c,f)+BLOCK-MULTIPLY(d,h)
    return the four blocks assembled into C
```

索引 view 只保存起點、stride 與階數，可用常數時間分割。即使真的複製區塊，其 $\Theta(n^2)$ 成本也與固定個區塊加法同階，因此 $T(n) = 8T(n/2) + \Theta(n^2)$，基例為 $\Theta(1)$。

Strassen 的七個乘積全部列出

以下各乘積由遞迴 STRASSEN 計算；括號內的加減是逐元素的區塊運算。

$$
\begin{aligned}
M_1 &= (a+d)(e+h), & M_2 &= (c+d)e, \\
M_3 &= a(f-h), & M_4 &= d(g-e), \\
M_5 &= (a+b)h, & M_6 &= (c-a)(e+f), \\
M_7 &= (b-d)(g+h). &&
\end{aligned}
$$

再組合四個結果：

$$
\begin{aligned}
C_{11} &= M_1 + M_4 - M_5 + M_7, & C_{12} &= M_3 + M_5, \\
C_{21} &= M_2 + M_4, & C_{22} &= M_1 - M_2 + M_3 + M_6.
\end{aligned}
$$

$n = 1$ 時直接做純量乘法；$n > 1$ 時依此分割、計算七個乘積，再組成四塊。這些步驟與基例一起定義完整遞迴程序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

四個輸出區塊為何正確

只用分配律與加法消去，保持矩陣乘法的因子順序：

$$
\begin{aligned}
C_{11} &= (ae + ah + de + dh) + (dg - de) \\
&\quad - (ah + bh) + (bg + bh - dg - dh) = ae + bg, \\
C_{12} &= (af - ah) + (ah + bh) = af + bh, \\
C_{21} &= (ce + de) + (dg - de) = ce + dg, \\
C_{22} &= (ae + ah + de + dh) - (ce + de) \\
&\quad + (af - ah) + (ce + cf - ae - af) = cf + dh.
\end{aligned}
$$

基例的純量乘法正確。若七個較小乘積均由歸納假設正確算出，上述恆等式便組成 AB 的全部四塊，完成歸納證明。

這裡需要加法、減法與分配律。只有加法、沒有減法的一般半環（semiring），不能直接套用這七式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

七乘積的數值與四塊回收

取 $A = \begin{bmatrix} 2 & -1 \\ 3 & 4 \end{bmatrix}$、$B = \begin{bmatrix} 5 & 2 \\ -2 & 1 \end{bmatrix}$。

乘積	代入	值
M_1	$(2+4)(5+1)$	36
M_2	$(3+4)5$	35
M_3	$2(2-1)$	2
M_4	$4(-2-5)$	-28
M_5	$(2-1)1$	1
M_6	$(3-2)(5+2)$	7
M_7	$(-1-4)(-2+1)$	5

Lin Cing https://github.com/xinhi6716/computer-science-self-study

七乘積的數值與四塊回收：圖解

數例的四塊位置：C11、C12、C21、C22 不可互換

A		×	B		=	C	
2	-1		5	2		12	3
3	4		-2	1		7	10

C11: 36−28−1+5=12

C12: 2+1=3

C21: 35−28=7

C22: 36−35+2+7=10

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

直接乘法核對相同結果：

$$AB = \begin{bmatrix} 10+2 & 4-1 \\ 15-8 & 6+4 \end{bmatrix} = \begin{bmatrix} 12 & 3 \\ 7 & 10 \end{bmatrix}.$$

Strassen 的時間、補零與記憶體

Strassen 的七個子乘積需要 10 次輸入區塊加減，組合四個結果還需 8 次。view 分割為 $O(1)$，所以

$$T(n) = 7T(n/2) + \Theta(n^2) = \Theta(n^{\lg 7}), \qquad \lg 7 \approx 2.80735.$$

加減的常數較大，小矩陣未必較快；可在固定門檻改用標準乘法，不改漸近指數。代數恆等式的正確性也不等於浮點誤差保證。

任意 $n \geq 1$ 可補零到 $m = 2^{\lceil \lg n \rceil}$，$n \leq m < 2n$。左上 $n \times n$ 結果仍是原乘積，且 $m^{\lg 7} < 7n^{\lg 7}$，故同階。空矩陣直接返回，不遞迴。

依序完成七個子呼叫、適時釋放暫存時，每層只保留固定個當層區塊，峰值空間滿足

$$S(n) \leq S(n/2) + O(n^2) = O(n^2).$$

另有 $O(\lg n)$ 個框架。若同時保留所有子問題工作區，就必須重建空間遞迴式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

延伸工具：Akra–Bazzi 的適用條件

這裡引用 MIT *Mathematics for Computer Science*（2010）§10.4.3 的定理，採用正且有界的基例作充分條件；**不把下列應用當成定理的完整證明**。

$$T(x) = \sum_{i=1}^{k} a_i T(b_i x + h_i(x)) + g(x), \qquad x > x_0.$$

- k 有限固定，$a_i > 0$、$0 < b_i < 1$ 固定。
- $0 \le x \le x_0$ 時，固定常數 $0 < d_1 \le T(x) \le d_2$。
- $g \ge 0$ 可微，$|g'(x)|$ 至多以多項式成長；$|h_i(x)| = O(x/\log^2 x)$。
- x_0 足夠大，所有子引數都在 $[0, \rho x]$，固定 $\rho < 1$，使遞迴有定義且會停止。

上述條件是本章使用的版本。若基例與 g 全為零，成本恆零，就不能得到帶正葉成本的結論。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

先求唯一指數，再做積分

令 p 滿足 $\sum_i a_i b_i^p = 1$，則引用定理給出

$$T(x) = \Theta\left(x^p \left[1 + \int_1^x \frac{g(u)}{u^{p+1}}\, du\right]\right).$$

因 $0 < b_i < 1$，函數 $F(p) = \sum_i a_i b_i^p$ 連續且嚴格遞減；當 $p \to -\infty$ 時趨向無窮大，$p \to +\infty$ 時趨向 0，所以恰有一個實根。

對 $T(x) = T(x/3) + T(2x/3) + x$，取固定小輸入為正常數，比例方程為 $(1/3)^p + (2/3)^p = 1$，故 $p = 1$。代入得

$$T(x) = \Theta\left(x \left[1 + \int_1^x \frac{du}{u}\right]\right) = \Theta(x \log x).$$

這和前面用完整層與最深層得到的兩側界一致；兩種推導的適用前提都要保留。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

取整與比例改變，是兩種不同的事

floor、ceiling 與固定平移的誤差至多為常數，符合 $O(x/\log^2 x)$；仍須選夠大的基例範圍，確保子問題縮小。

但 $T(n) = T(n/2 + n/4) + T(n/2) + n$ 的第一個子問題其實是 $3n/4$。$n/4$ 不符合小擾動條件；不能沿用兩個 $1/2$ 比例。

正確方程為 $(3/4)^p + (1/2)^p = 1$，唯一根 $p \approx 1.507126591639 > 1$。由

$$\int_1^n u^{-p}\,du = \frac{1 - n^{1-p}}{p-1} = \Theta(1),$$

得到 $T(n) = \Theta(n^p)$。嚴格指數由方程定義，小數只是近似值。

若其他教材把子問題寫成 n/c_i，則本章 $b_i = 1/c_i$。轉換表示法時，方程也必須一起換。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

延伸閱讀與定理來源

- Cormen、Leiserson、Rivest、Stein，*Introduction to Algorithms*, 3rd Edition：Chapter 2（合併排序）、Chapter 4（分治、代入法、遞迴樹、主定理及其證明）。
- Demaine、Leiserson，MIT 6.046J（Fall 2005），Lecture 2–3：遞迴式，以及二分搜尋、快速冪、Fibonacci 與矩陣乘法的連結。
- Lehman、Leighton、Meyer，*Mathematics for Computer Science*（2010），§ 10.4.3，Theorem 10.4.1：MIT 官方講義。本章使用正且有界基例的充分條件版本。

主定理的三情形在正文給出證明；Akra–Bazzi 為引用的延伸定理，正文負責核對條件、求指數與計算積分。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study